Documentation: I did not collaborate with any other students on this assignment. I received help with adjusting my code from CF Sebastian Hurubaru and Richard Wang.

2.ci

LinUCB, 5 seeds, no new arms.
Mean = 0.569
Standard deviation = 0.077

2.cii

LinUCB, 5 seeds, popular strategy.
$K \in 500, 1000, 2500, 5000$.
$K = 500$: Mean = 0.34, Std dev = 0.05
$K = 1000$: Mean = 0.421, Std dev = 0.067
$K = 2500$: Mean = 0.491, Std dev = 0.077
$K = 5000$: Mean = 0.532, Std dev = 0.077
The results improve as K increases, but are worse than without adding any new arms. This is likely because for low K, the algorithm is not choosing the best arms to explore based on the simple selection strategy.

2.ciii

LinUCB, 5 seeds, corrective strategy.
$K \in 500, 1000, 2500, 5000$.
$K = 500$: Mean = 0.34, Std dev = 0.05
$K = 1000$: Mean = 0.421, Std dev = 0.067
$K = 2500$: Mean = 0.491, Std dev = 0.077
$K = 5000$: Mean = 0.532, Std dev = 0.077
With the same simulator inputs, the corrective and popular update strategies produce the same results. Both methods introduce arms that are close to existing popular arms, so this could be an artifact of low arm count and low K.

2.civ

LinUCB, 5 seeds, counterfactual strategy.
$K \in 500, 1000, 2500, 5000$.
$K = 500$: Mean = 0.178, Std dev = 0.06
$K = 1000$: Mean = 0.268, Std dev = 0.08
$K = 2500$: Mean = 0.357, Std dev = 0.097
$K = 5000$: Mean = 0.435, Std dev = 0.081

Strategy	Mean	Standard Deviation
No new arms	0.569	0.077
Popular ($K = 5000$)	0.532	0.077
Corrective ($K = 5000$)	0.532	0.077
Counterfactual ($K = 5000$)	0.435	0.081

Table 1: Simulated Strategy Comparison

The popular and corrective arm strategies are better than the counterfactual strategy which underperforms, especially for low K. None of the three strategies are better than the case where we do not add new arms. This is likely due to low K and overfitting, or more likely due to a counterfactual implementation error as I discuss below.

2.cv

My plot does not look the results provided.

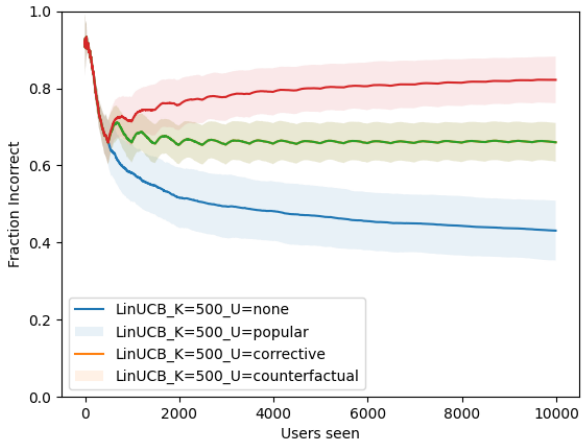

Figure 1: Users Seen vs. Fraction Incorrect

It is appears that the most glaring difference exists in the results of the counterfactual strategy. Though my code passes the autograder, I seem to have a much higher fraction incorrect for the counterfactual strategy, especially for low users seen. The different strategies perform modestly for low K, expect the counterfactual should actually be outperforming the blue LinUCB method with no arms added. This is because the counterfactual strategy is more aggressive in its corrections using gradient ascent to minimize regret for low user interactions.

2.cvi

Again, my plot does not identically resemble the results provided.

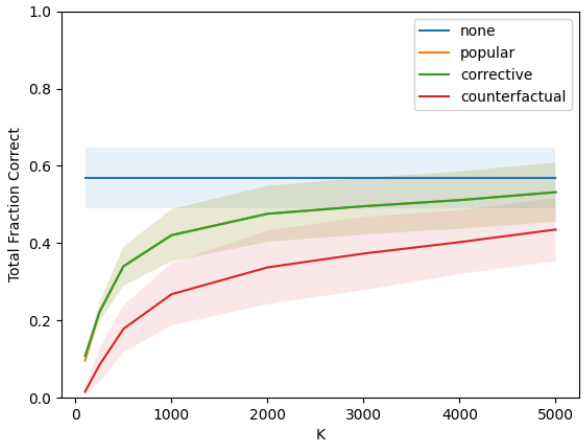

Figure 2: K vs. Total Fraction Correct

The popular and corrective strategies improve as you increase K, while the counterfactual strategy gets worse. This is because the popular strategy gains a more stable estimate of the best arms over more steps, and the corrective strategy averages over better arms and a broader range of errors for high K. Conversely, the counterfactual strategy should encounter overfitting when adjusting the arm over too many data points.

2.d

Scenarios 2, 3, and 4 each have their own ethical implications, but I think that scenario 3 where the algorithm can specifically be used to influence the user's dynamic preferences is the most alarming. To understand whether an algorithm is vectoring user preferences over time, it would be advantageous to store and observe a user's algorithmic history to determine whether current preferences were catalyzed by non-static recommendations at any point. Personally, I doubt that this data history is usable even if it is cached for certain forms of content because scenario 3 could emerge from nebulous recommendations at scale.

2.e

In my opinion, the platforms are responsible for identifying the negative feedback loop, but only insofar as protecting their creators. For example, scenario 1 seems to be a fairly cut-and-dry case where the static preferences given by a model will likely grow boring over time. Depending on the medium of the content, the creators are likely less agile in their ability to react to trends than the platform developers. Likewise, scenario 2 presents an issue with attracting more creators to the platform, limiting long-term platform success. Scenarios 3 and 4 are more ethically concerning to the user, but users should be responsible for breaking their own negative feedback loops. Scenario 4 is presented as a sort of remedy to the dynamic preference influence of scenario 3. User freedom should mean that they have the responsibility for recognizing algorithmic shifts, the ability to create looping kinds, and ultimately the choice to remove themselves from the feedback mechanism.